HDOC Day-46

Write a C program that reads the *size* and elements of a variable-length integer array, *nums[]*. The array may contain duplicate elements. Pass the array to a function named *Bubble_sort()* using its address and sort the elements in ascending order using the Bubble Sort technique. After sorting, pass the sorted array to another function named *occurrence()*. Use pointers to pass the array between functions. The function *occurrence()* must:

1. Read an integer *target*.
2. Find the first and last occurrences of *target* in the sorted array.
3. Display the values and indices of the first and last occurrences.
4. Display -1, -1 if the *target* is not present.

Example 1:

Array: 5 7 7 8 8 10

Target: 8

Output:

First occurrence: 8 at index 3

Last occurrence: 8 at index 4

Example 2:

Array: 5 7 7 8 10

Target: 8

Output:

First occurrence: 8 at index 3

Last occurrence: 8 at index 3

Example 3:

Array: 5 7 7 8 8 10

Target: 6

Output:

-1, -1

1. Algorithm

1. Start.
2. Read the size n of the array.
3. Read the n integer elements into the variable-length array nums[n].
4. Pass the array and its size to Bubble_sort() using pointers.
5. Sort the array in ascending order using Bubble Sort.
6. Pass the sorted array to occurrence().
7. Read the target element from the user.
8. Initialize first = -1 and last = -1.
9. Traverse the sorted array:
    - If an element equals the target and first == -1, store its index in first.
    - Whenever an element equals the target, update last with its index.
10. If the target is found, display its first and last occurrence with their indices.
11. Otherwise, display -1, -1.
12. Stop.

2. C program

```
#include <stdio.h>


void Bubble_sort(int *arr, int n)
{
   int i, j, temp;


   for (i = 0; i < n - 1; i++)
   {
      for (j = 0; j < n - i - 1; j++)
```

```
      {
         if (*(arr + j) > *(arr + j + 1))
         {
            temp = *(arr + j);
            *(arr + j) = *(arr + j + 1);
            *(arr + j + 1) = temp;
         }
      }
   }
}

void occurrence(int *arr, int n)
{
   int target, first = -1, last = -1;
   int i;

   printf("Enter target: ");
   scanf("%d", &target);

   for (i = 0; i < n; i++)
   {
      if (*(arr + i) == target)
      {
         if (first == -1)
         {
            first = i;
         }
```

```
            last = i;
        }
    }

    if (first != -1)
    {
        printf("First occurrence: %d at index %d\n",
            target, first);
        printf("Last occurrence: %d at index %d\n",
            target, last);
    }
    else
    {
        printf("-1, -1\n");
    }
}

int main()
{
    int n, i;

    printf("Enter array size: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
```

```
    }

    int nums[n];

    printf("Enter array elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    Bubble_sort(nums, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", nums[i]);
    }
    printf("\n");

    occurrence(nums, n);

    return 0;
}
```

Test Case Table:

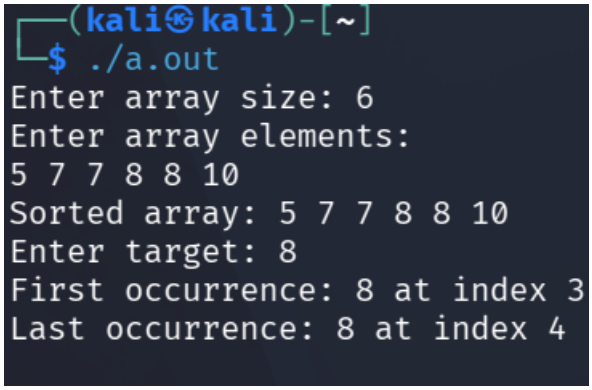
Test Case	Input array	Target	Expected Output
1	5 7 7 8 8 10	8	First 8 at index 3; Last 8 at index 4
2	5 7 7 8 10	8	First: 8 at index 3; Last 8 at index 3
3	5 7 7 8 8 10	6	-1, -1

4	4 4 4 4	4	First: 4 at index 0; Last: 4 at index 3
5	9	9	First: 9 at index 0; 9 at index 0

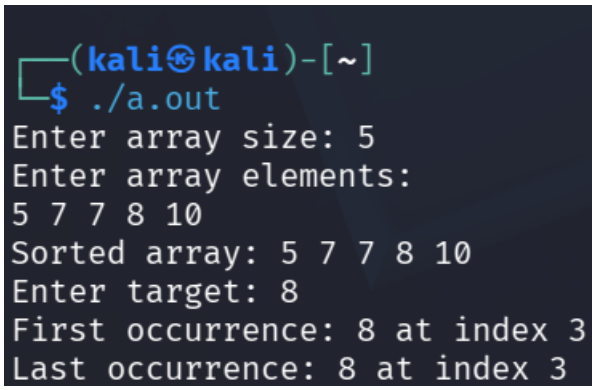
Compilation and Execution:

The following program has been compiled and executed successfully.

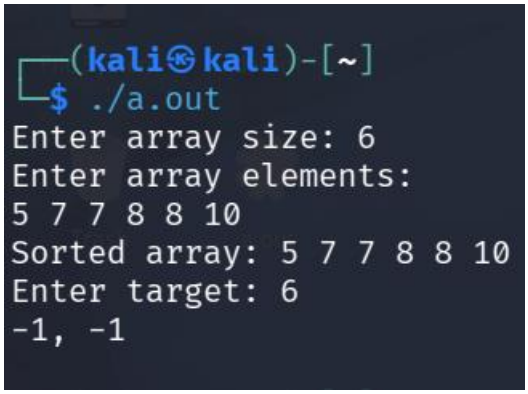
Test Case 1:



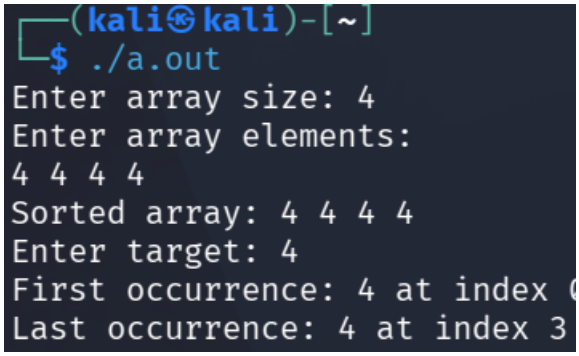
Test Case 2:



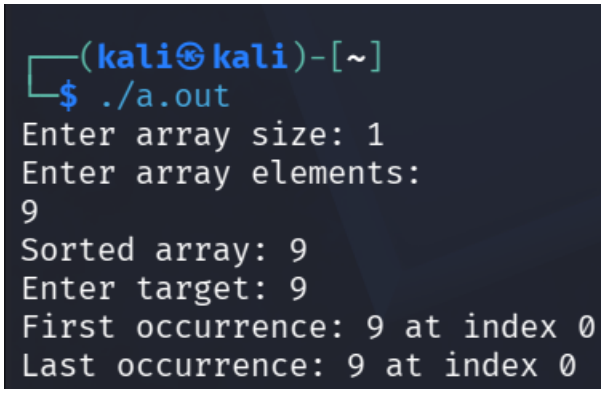
Test Case 3:



Test Case 4:


(kali㉿kali)-[~]
$ ./a.out
Enter array size: 4
Enter array elements:
4 4 4 4
Sorted array: 4 4 4 4
Enter target: 4
First occurrence: 4 at index
Last occurrence: 4 at index 3

Test Case 5:


(kali㉿kali)-[~]
$ ./a.out
Enter array size: 1
Enter array elements:
9
Sorted array: 9
Enter target: 9
First occurrence: 9 at index 0
Last occurrence: 9 at index 0